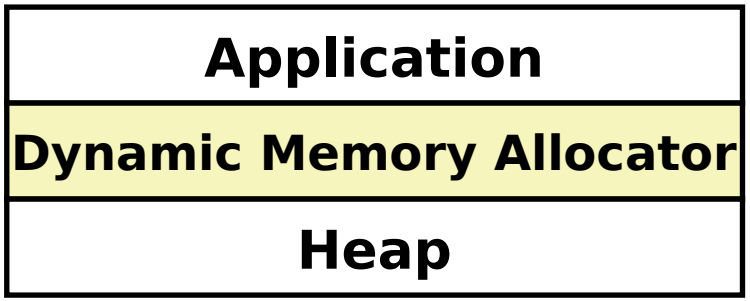
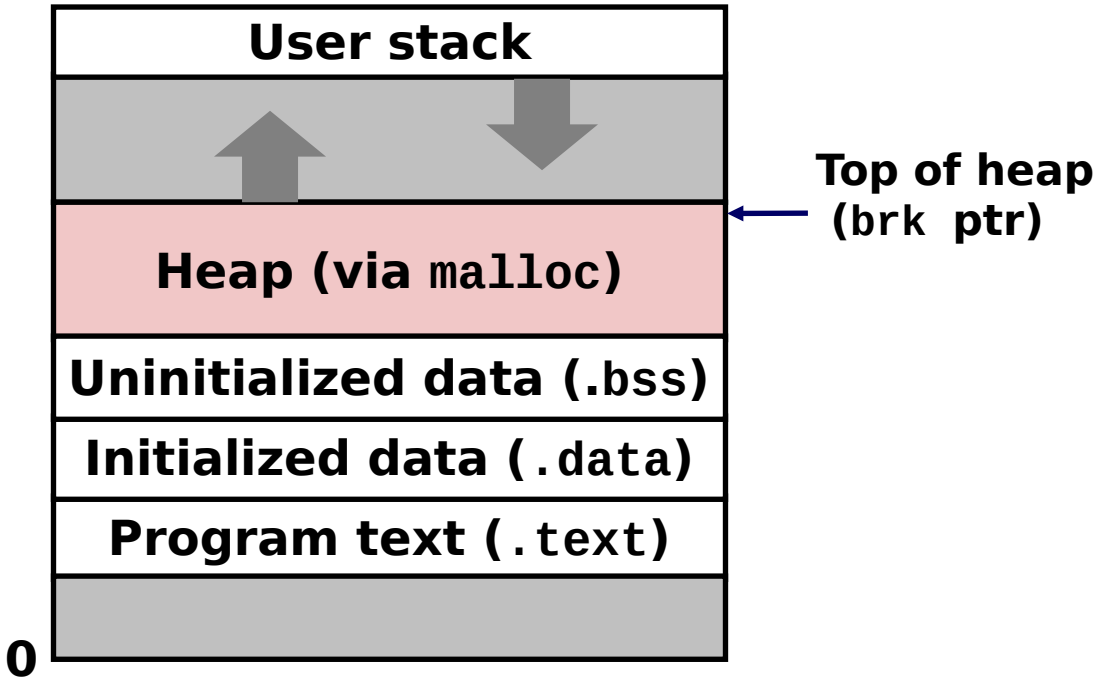
Dynamic Memory Allocation

- **Programmers use *dynamic memory allocators* (such as `malloc`) to acquire VM at run time.**
  - For data structures whose size is only known at runtime.



- **Dynamic memory allocators manage an area of process virtual memory known as the *heap*.**



Dynamic Memory Allocation

- **Allocator maintains heap as collection of variable sized *blocks*, which are either *allocated* or *free***
- **Types of allocators**
  - ***Explicit allocator*:** application allocates and frees space
    - E.g., `malloc` and `free` in C
  - ***Implicit allocator:*** application allocates, but does not free space
    - E.g. garbage collection in Java, ML, and Lisp

Allocation Example

Assumptions:

- **Memory is word addressed.**
- **Words are int-sized.**
- **Allocations are double-word aligned.**

```
#define SOI sizeof(int)
```

```
p1 = malloc(4*SOI)
```



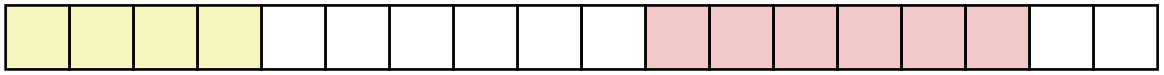
```
p2 = malloc(5*SOI)
```



```
p3 = malloc(6*SOI)
```



```
free(p2)
```



```
p4 = malloc(2*SOI)
```

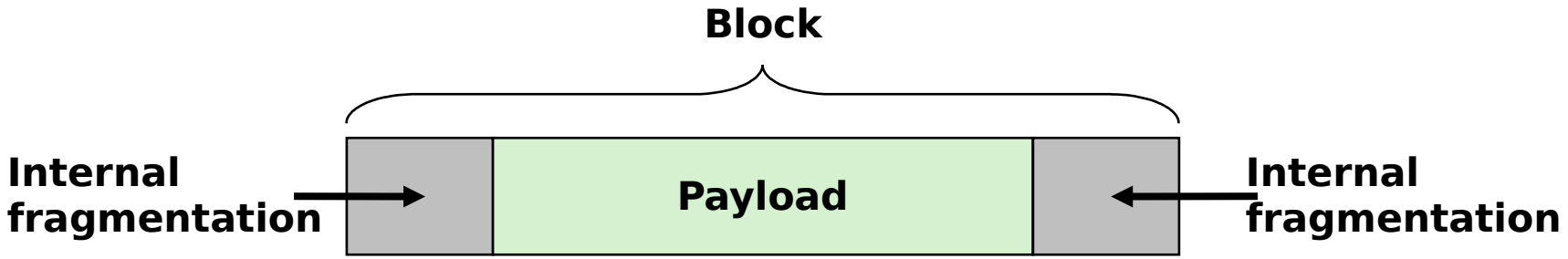


Constraints

- **Applications**
  - Can issue arbitrary sequence of **`malloc`** and **`free`** requests
  - **`free`** request must be to a **`malloc`**'d block
- **Allocators**
  - Can't control number or size of allocated blocks
  - Must respond immediately to **`malloc`** requests
    - *i.e.*, can't reorder or buffer requests
  - Must allocate blocks from free memory
    - *i.e.*, can only place allocated blocks in free memory
  - Must align blocks so they satisfy all alignment requirements
    - 8-byte (x86) or 16-byte (x86-64) alignment on Linux boxes
  - Can manipulate and modify only free memory
  - Can't move the allocated blocks once they are **`malloc`**'d
    - *i.e.*, compaction is not allowed

Internal Fragmentation

- **For a given block, *internal fragmentation* occurs if payload is smaller than block size**



- **Caused by**
  - Overhead of maintaining heap data structures
  - Padding for alignment purposes
  - Explicit policy decisions
    (e.g., to return a big block to satisfy a small request)
- **Depends only on the pattern of *previous* requests**
  - Thus, easy to measure

External Fragmentation

```
#define SOI sizeof(int)
```

- **Occurs when there is enough aggregate heap memory, but no single free block is large enough**

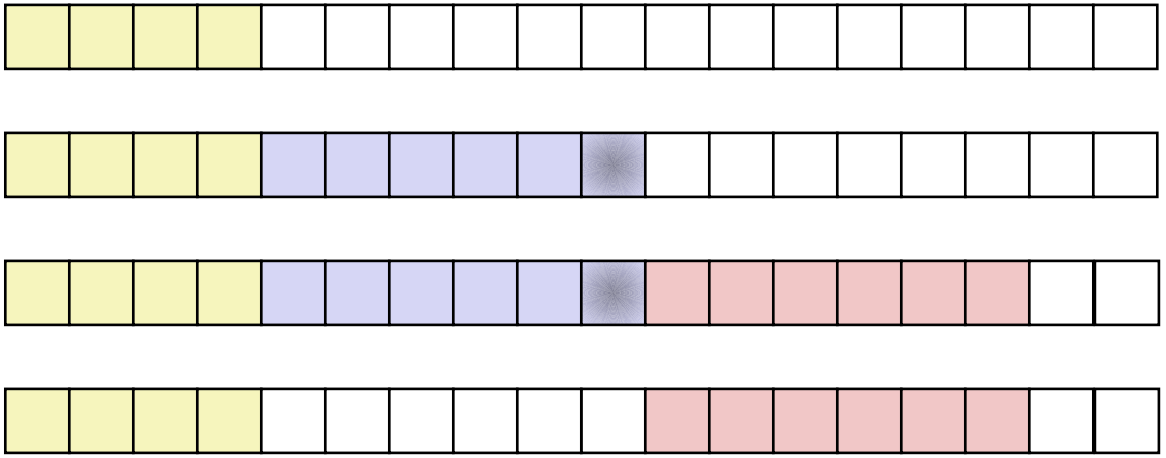
```
p1 = malloc(4*SOI)
```

```
p2 = malloc(5*SOI)
```

```
p3 = malloc(6*SOI)
```

```
free(p2)
```

```
p4 = malloc(7*SOI)
```



Oops! (what would happen now?)

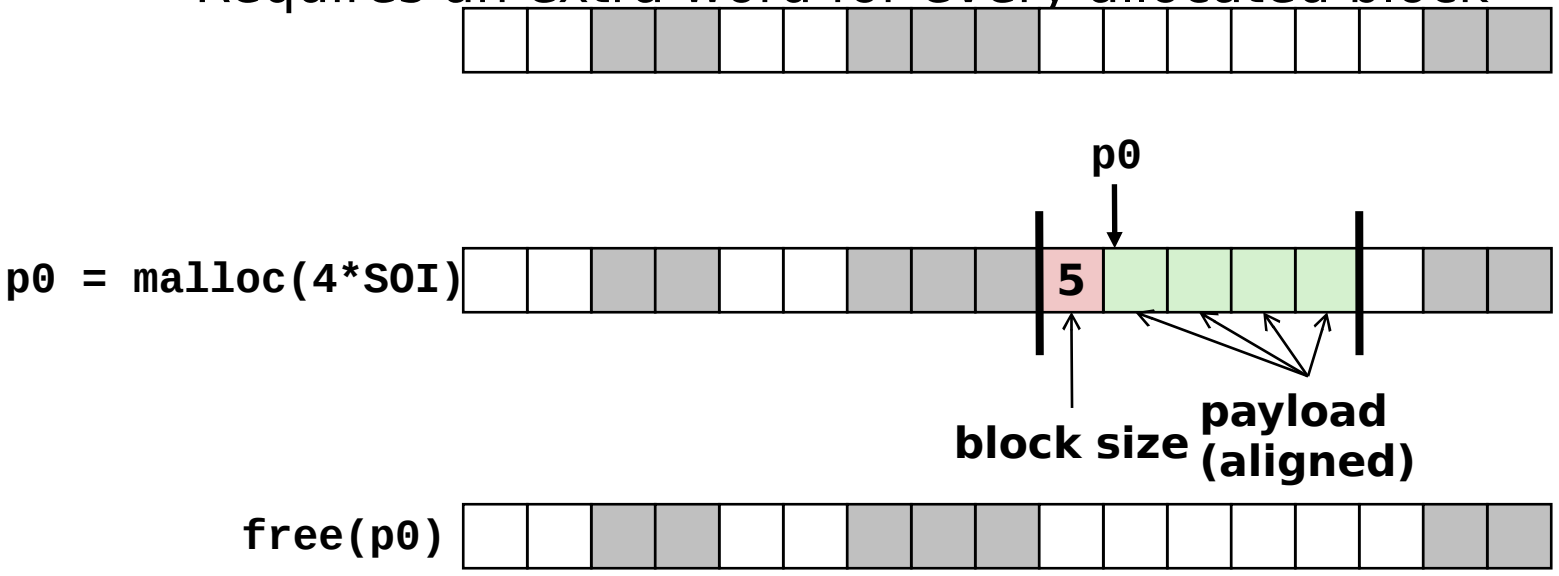
- **Depends on the pattern of future requests**
  - Thus, difficult to measure

Implementation Issues

- **How do we know how much memory to free given just a pointer?**
- **How do we keep track of the free blocks?**
- **What do we do with the extra space when allocating a structure that is smaller than the free block it is placed in?**
- **How do we pick a block to use for allocation -- many might fit?**
- **How do we reinsert freed block?**

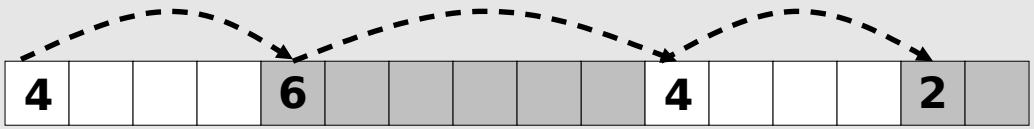
Knowing How Much to Free

- **Standard method**
  - Keep the length of a block in the word preceding the block.
    - This word is often called the ***header field*** or ***header***
  - Requires an extra word for every allocated block



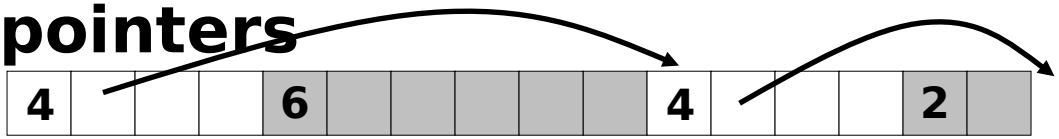
Keeping Track of Free Blocks

- **Method 1: *Implicit list* using length—links all blocks**



Need to tag each block as allocated/free

- **Method 2: *Explicit list* among the free blocks using pointers**



Need space for pointers

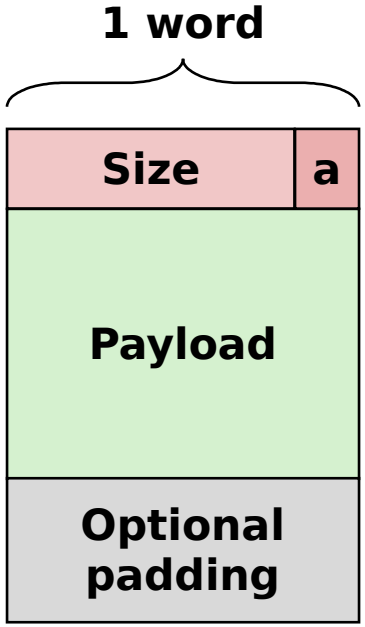
- **Method 3: *Segregated free list***
  - Different free lists for different size classes

- **Method 4: *Blocks sorted by size***
  - Can use a balanced tree (e.g. Red-Black tree) with pointers within each free block, and the length used as a key

Method 1: Implicit List

- **For each block we need both size and allocation status**
  - Could store this information in two words: wasteful!
- **Standard trick**
  - If blocks are aligned, some low-order address bits are always 0
  - Instead of storing an always-0 bit, use it as a allocated/free flag
  - When reading size word, must mask out this bit

Format of allocated and free blocks

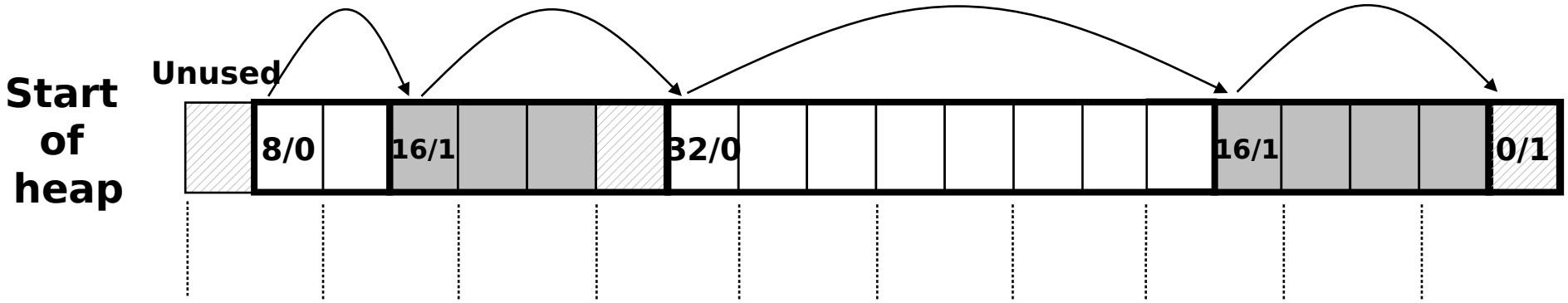


a = 1: Allocated block
a = 0: Free block

Size: block size

Payload: application data (allocated blocks only)

Detailed Implicit Free List Example



Double-word aligned

Allocated blocks: shaded
Free blocks: unshaded
Headers: labeled with “size in words/allocated bit”

Implicit List: Finding a Free Block

- ***First fit:***
  - Search list from beginning, choose ***first*** free block that fits:
  - Can take linear time in total number of blocks (allocated and free)
  - In practice it can cause “splinters” at beginning of list
- ***Next fit:***
  - Like first fit, but search list starting where previous search finished
  - Should often be faster than first fit: avoids re-scanning unhelpful blocks
  - Some research suggests that fragmentation is worse
- ***Best fit:***
  - Search the list, choose the ***best*** free block: fits, with fewest bytes left over
  - Keeps fragments small—usually improves memory utilization
  - Will typically run slower than first fit

Example 5:

Determine the block sizes and header values that would result from the following sequence of `malloc` requests. Assumptions: (1) The allocator maintains double-word alignment and uses an implicit free list with the block format from Figure 9.35. (2) Block sizes are rounded up to the nearest multiple of 8 bytes.

Request	Block size (decimal bytes)	Block header (hex)
`malloc(2)`		
`malloc(9)`		
`malloc(15)`		
`malloc(20)`		

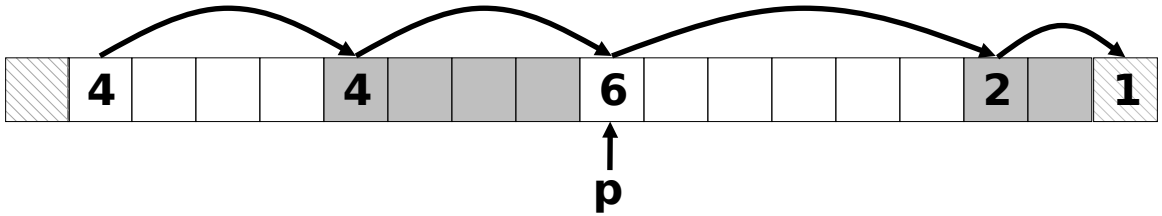
Example 5:

Determine the block sizes and header values that would result from the following sequence of `malloc` requests. Assumptions: (1) The allocator maintains double-word alignment and uses an implicit free list with the block format from Figure 9.35. (2) Block sizes are rounded up to the nearest multiple of 8 bytes.

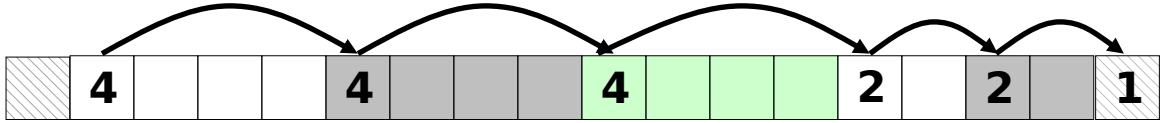
Request	Block size (decimal bytes)	Block header (hex)
`malloc(2)`	8	`0x9`
`malloc(9)`	16	`0x11`
`malloc(15)`	24	`0x19`
`malloc(20)`	24	`0x19`

Implicit List: Allocating in Free Block

- **Allocating in a free block: *splitting***
  - Since allocated space might be smaller than free space, we might want to split the block



`addblock(p, 4)`



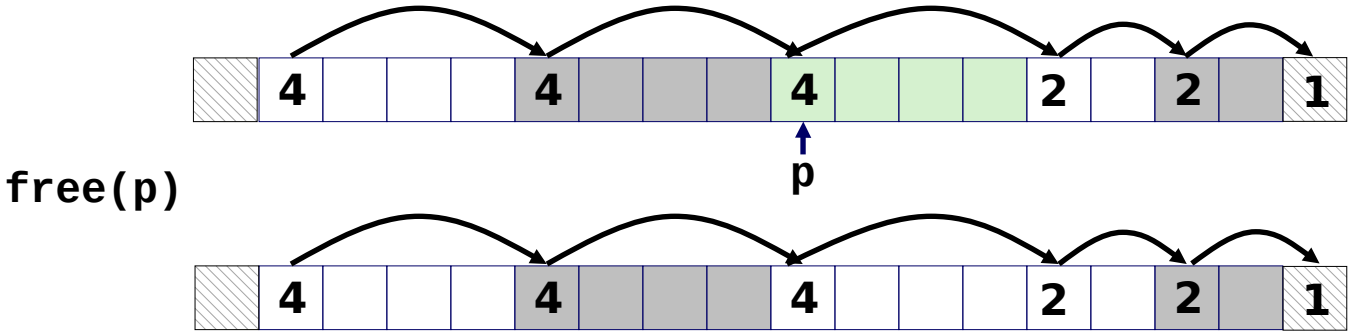

```
void addblock(ptr p, int len) {
  int newsize = ((len + 1) >> 1) << 1;  // round up to even
  int oldsize = *p & -2;                // mask out low bit
  *p = newsize | 1;                     // set new length
  if (newsize < oldsize)
    *(p+newsize) = oldsize - newsize;   // set length in remaining
}                                       //   part of block
```

Implicit List: Freeing a Block

- **Simplest implementation:**
  - Need only clear the “allocated” flag

```
void free_block(ptr p) { *p = *p & -2 }
```

  - But can lead to “false fragmentation”

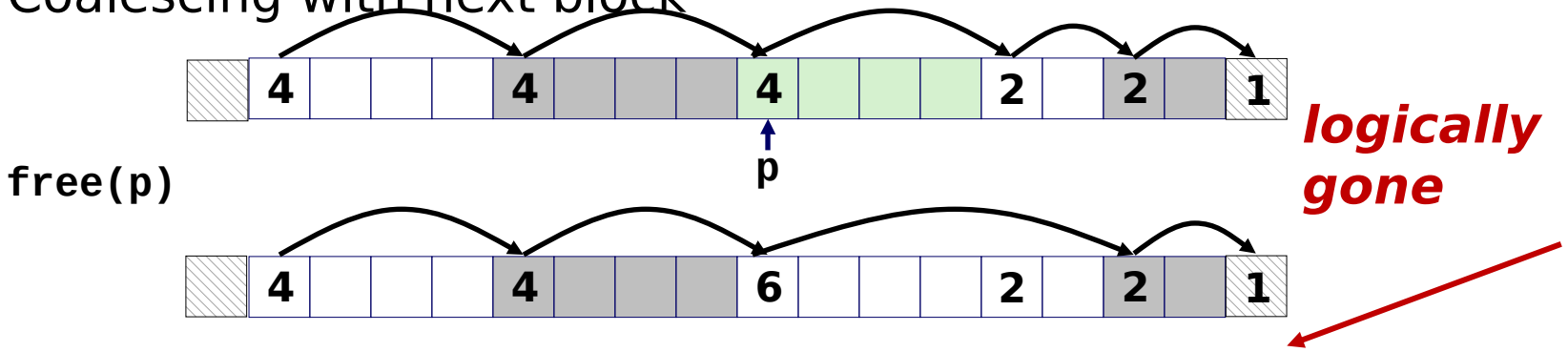


`malloc(5*SOI)` ***Oops!***

There is enough contiguous free space, but the allocator won’t be able to find it

Implicit List: Coalescing

- **Join *(coalesce)* with next/previous blocks, if they are free**
  - Coalescing with next block

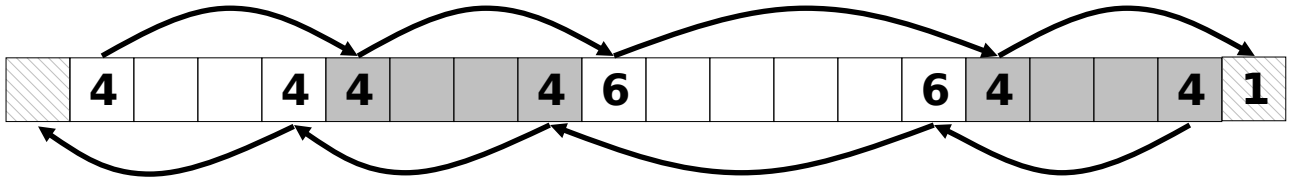



```
void free_block(ptr p) {
    *p = *p & -2;           // clear allocated flag
    next = p + *p;          // find next block
    if ((*next & 1) == 0)
      *p = *p + *next;      // add to this block if
}                           //    not allocated
```

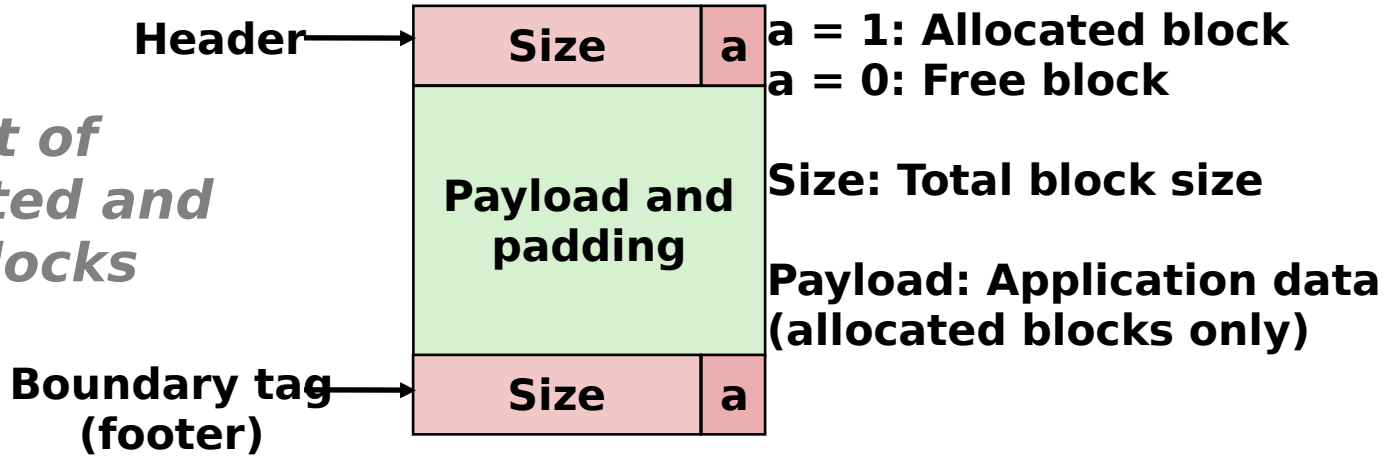
  - But how do we coalesce with *previous* block?

Implicit List: Bidirectional Coalescing

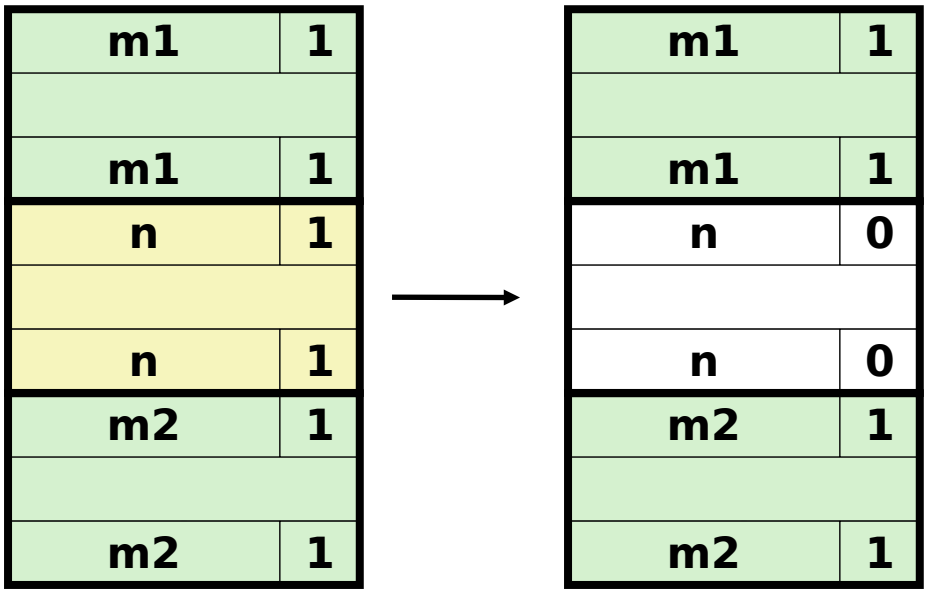
- ***Boundary tags*** [Knuth73]
  - Replicate size/allocated word at "bottom" (end) of free blocks
  - Allows us to traverse the "list" backwards, but requires extra space
  - Important and general technique!



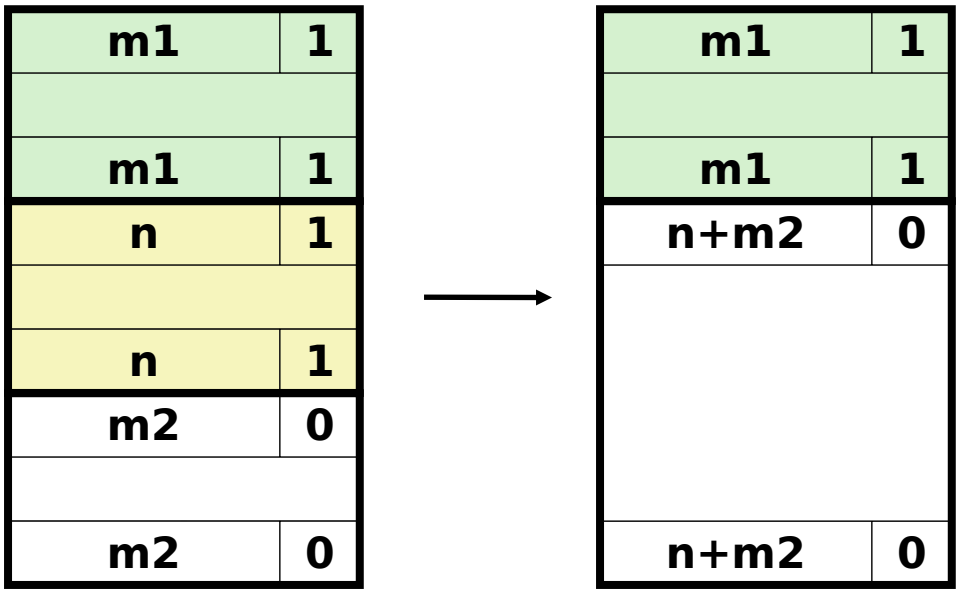
Format of allocated and free blocks



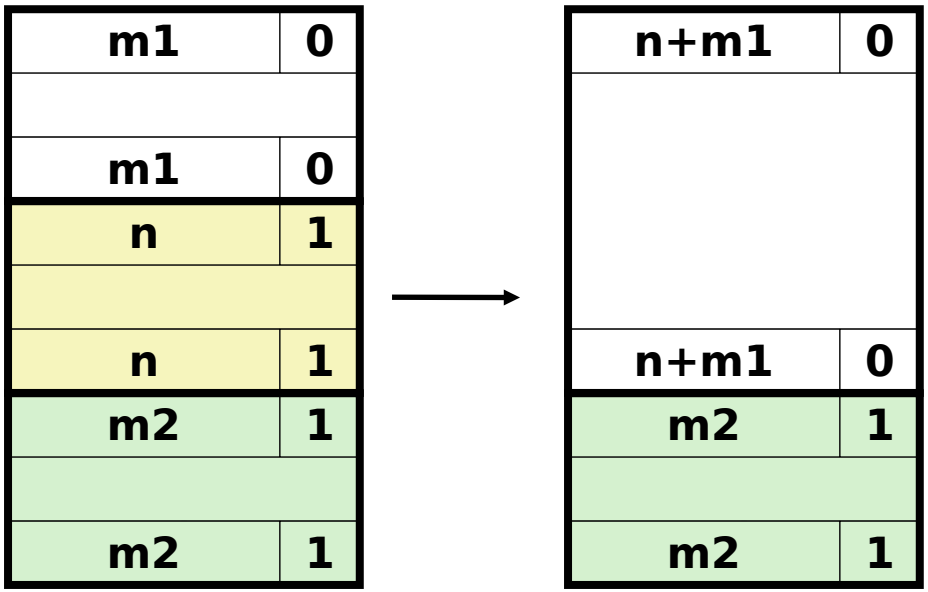
Constant Time Coalescing (Case 1)



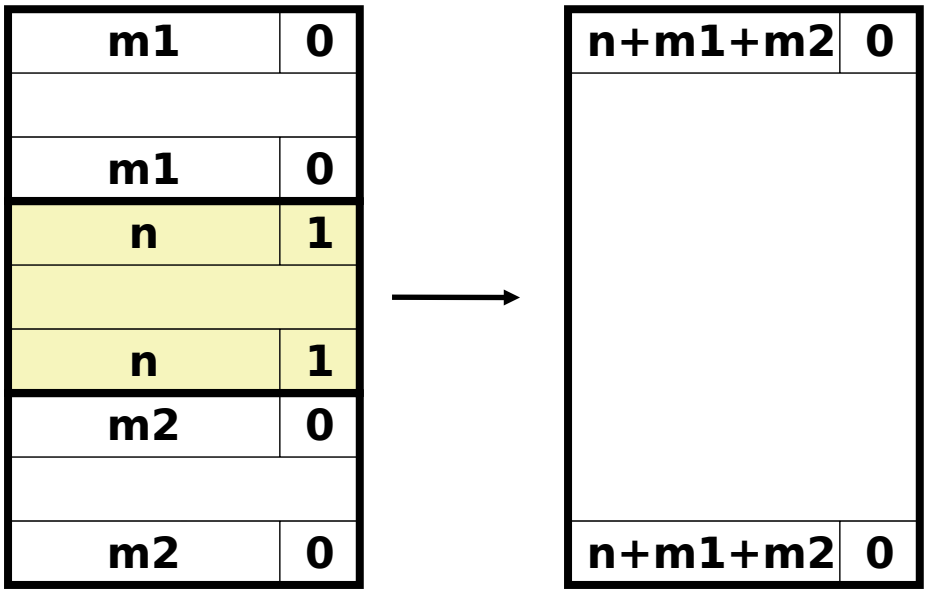
Constant Time Coalescing (Case 2)



Constant Time Coalescing (Case 3)



Constant Time Coalescing (Case 4)



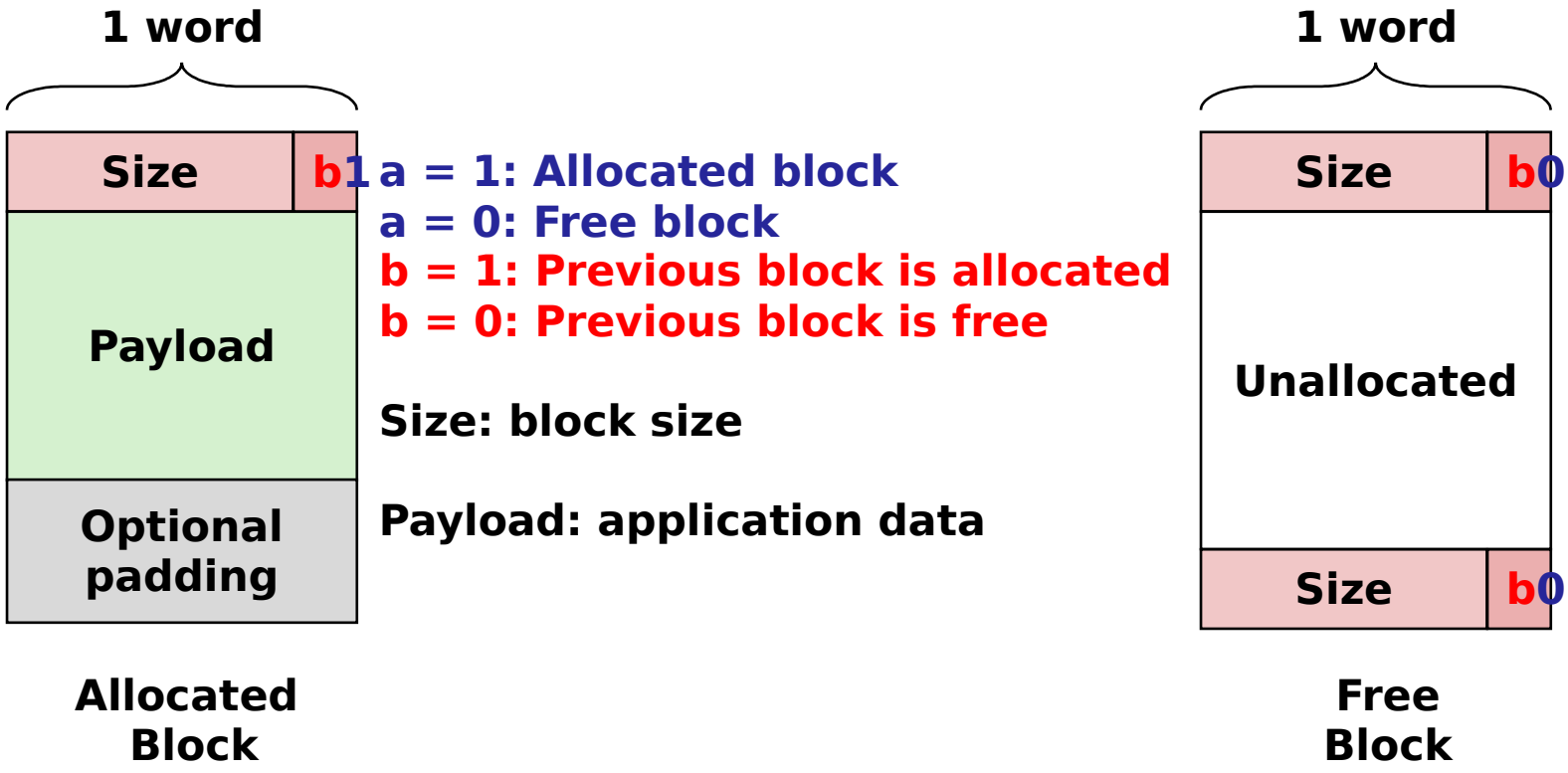
Disadvantages of Boundary Tags

- **Internal fragmentation**

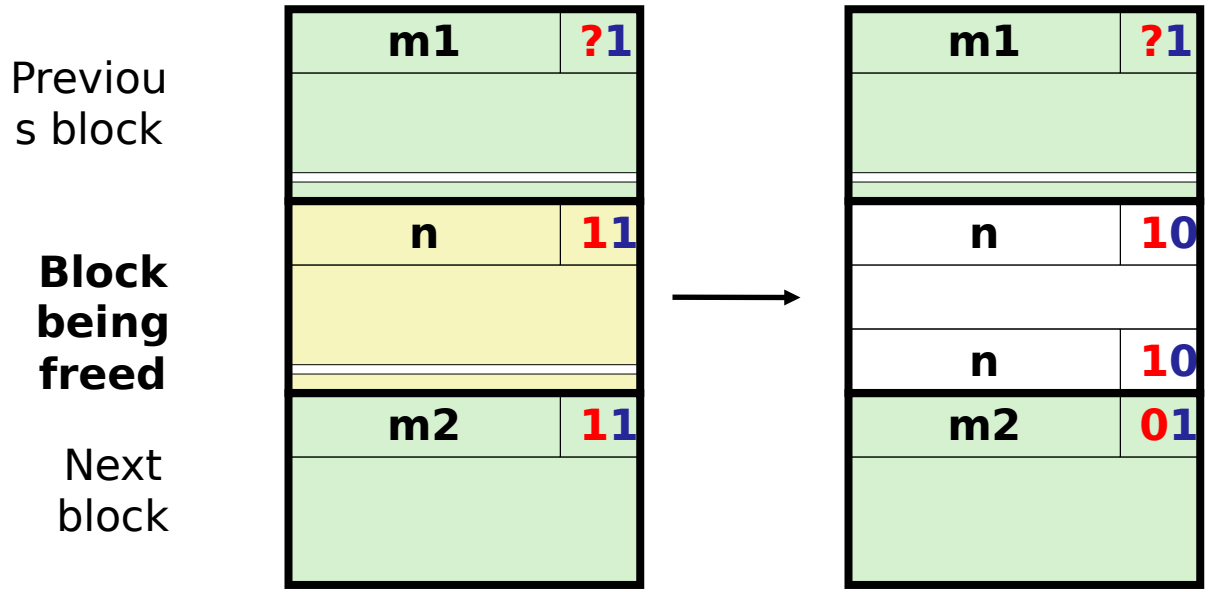
- **Can it be optimized?**
  - Which blocks need the footer tag?
  - What does that mean?

No Boundary Tag for Allocated Blocks

- Boundary tag needed only for free blocks
- When sizes are multiples of 4 or more, have 2+ spare bits

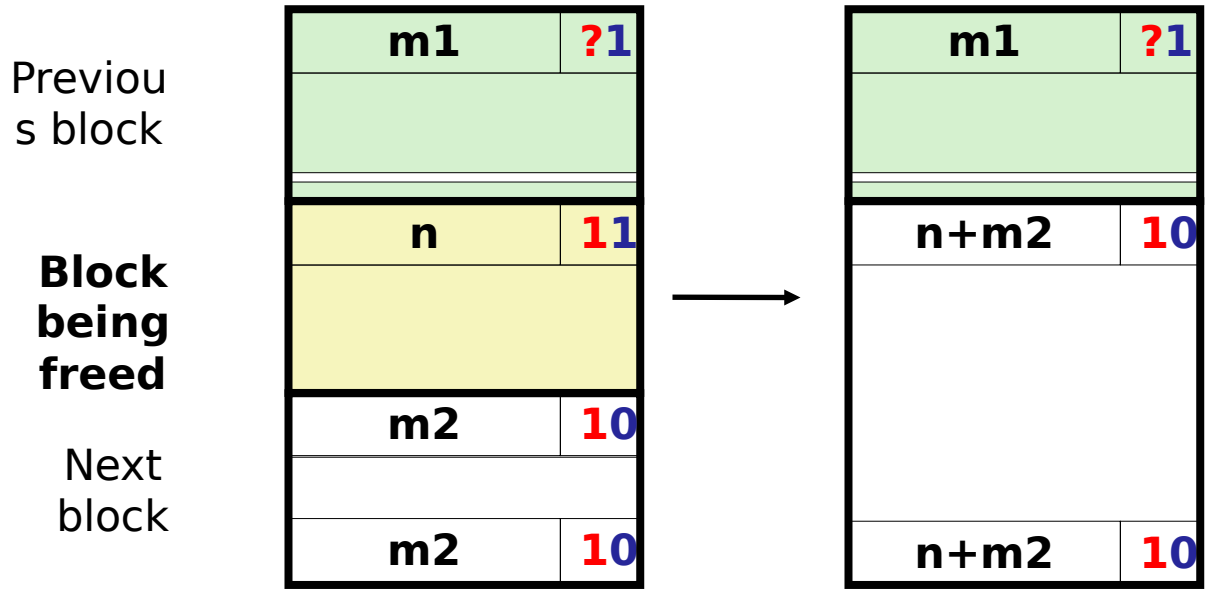


No Boundary Tag for Allocated Blocks (Case 1)



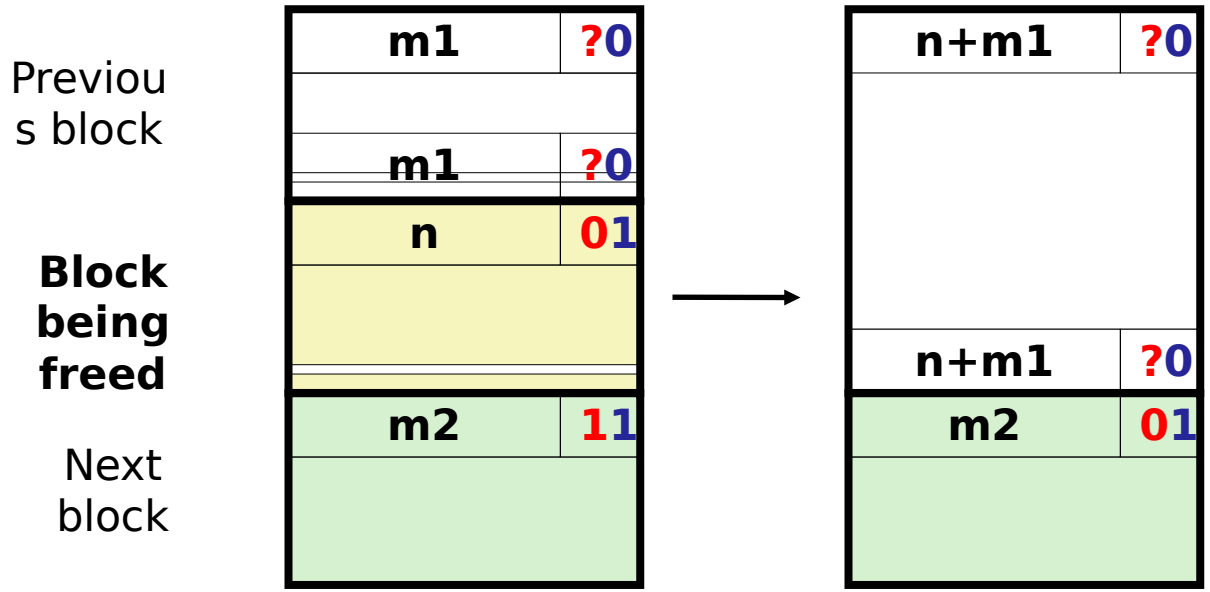
Header: Use 2 bits (always zero due to alignment):
(previous block allocated) << 1 | (current block allocated)

No Boundary Tag for Allocated Blocks (Case 2)



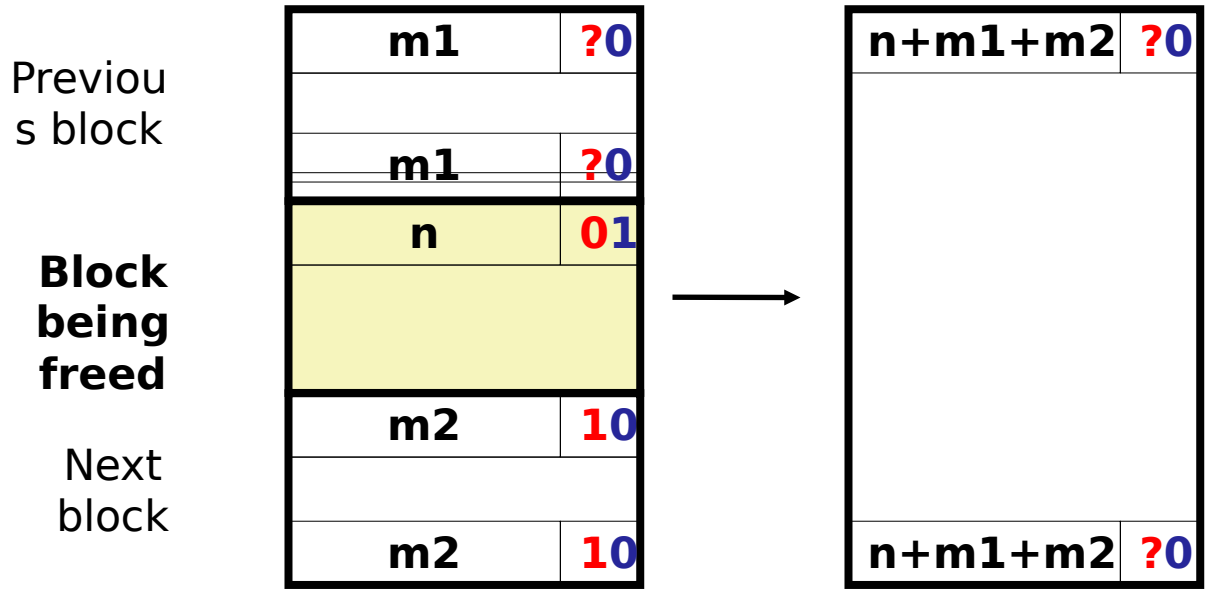
Header: Use 2 bits (always zero due to alignment):
(previous block allocated) << 1 | (current block allocated)

No Boundary Tag for Allocated Blocks (Case 3)



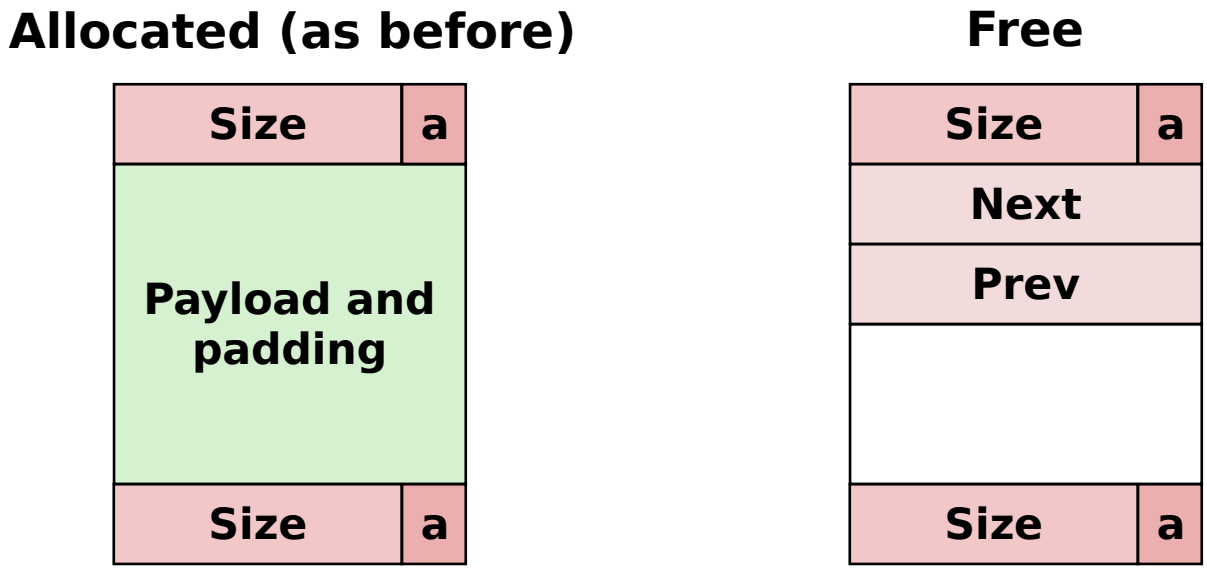
Header: Use 2 bits (always zero due to alignment):
(previous block allocated) << 1 | (current block allocated)

No Boundary Tag for Allocated Blocks (Case 3)



Header: Use 2 bits (always zero due to alignment):
(previous block allocated) << 1 | (current block allocated)

Explicit Free Lists



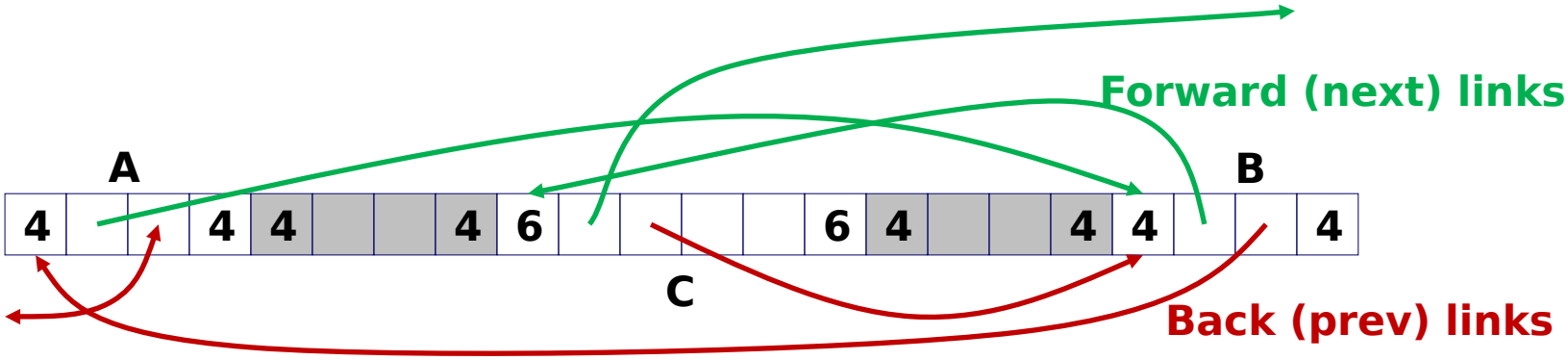
- **Maintain list(s) of *free* blocks, not *all* blocks**
  - The “next” free block could be anywhere
    - So we need to store forward/back pointers, not just sizes
  - Still need boundary tags for coalescing
  - Luckily we track only free blocks, so we can use payload area

Explicit Free Lists

- **Logically:**



- **Physically: blocks can be in any order**



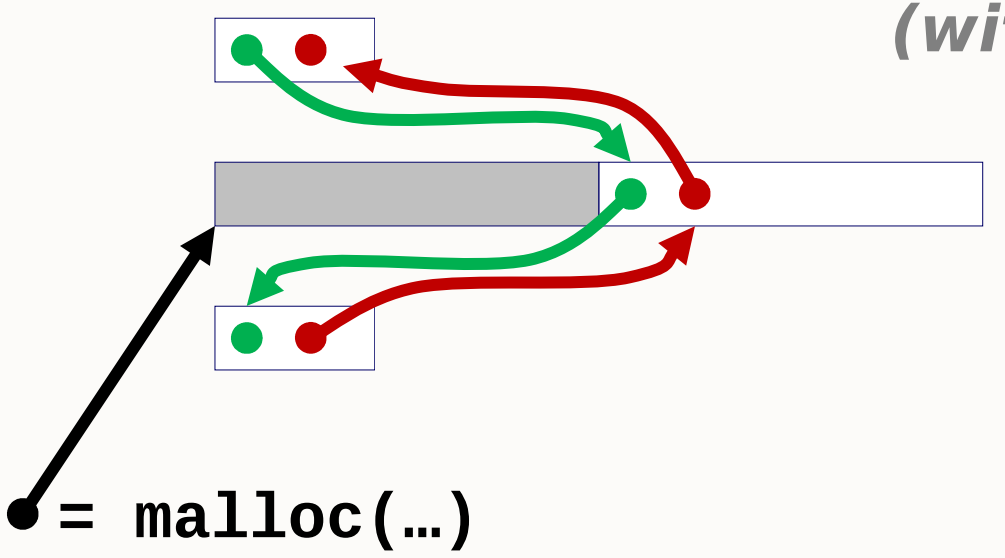
Allocating From Explicit Free Lists

conceptual graphic

Before



After *(with splitting)*



Freeing With Explicit Free Lists

- ***Insertion policy*: Where in the free list do you put a newly freed block?**
- **Unordered**
  - **LIFO (last-in-first-out) policy**
    - Insert freed block at the beginning of the free list
  - ***Pro:*** simple and constant time
  - ***Con:*** studies suggest fragmentation is worse than address ordered
- **Address-ordered policy**
  - Insert freed blocks so that free list blocks are always in address order:

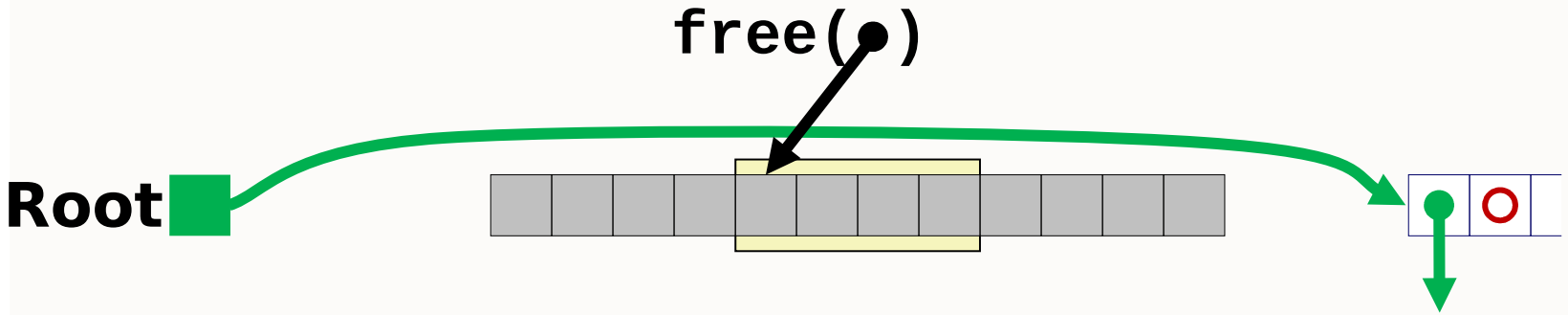
    $$addr(prev) < addr(curr) < addr(next)$$

  - ***Con:*** requires search
  - ***Pro:*** studies suggest fragmentation is lower than LIFO

Freeing With a LIFO Policy (Case 1)

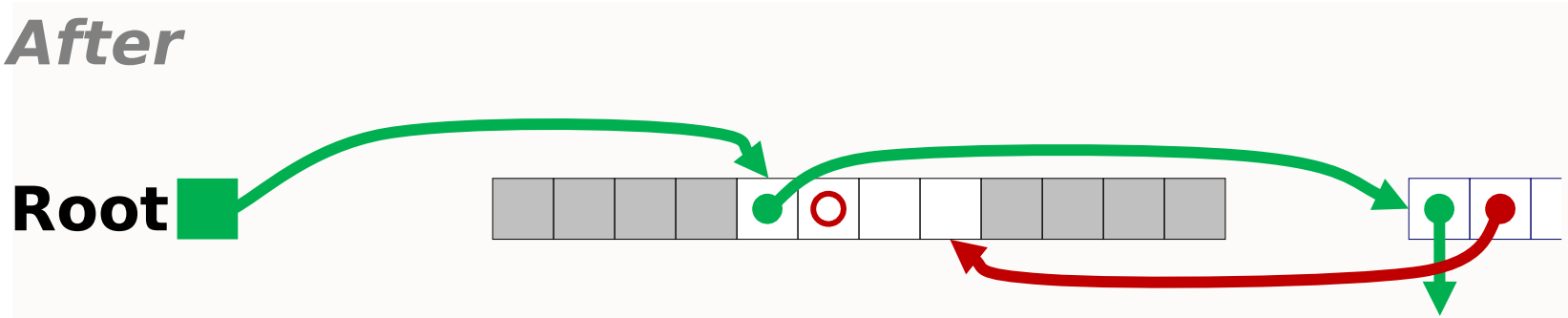
conceptual graphic

Before



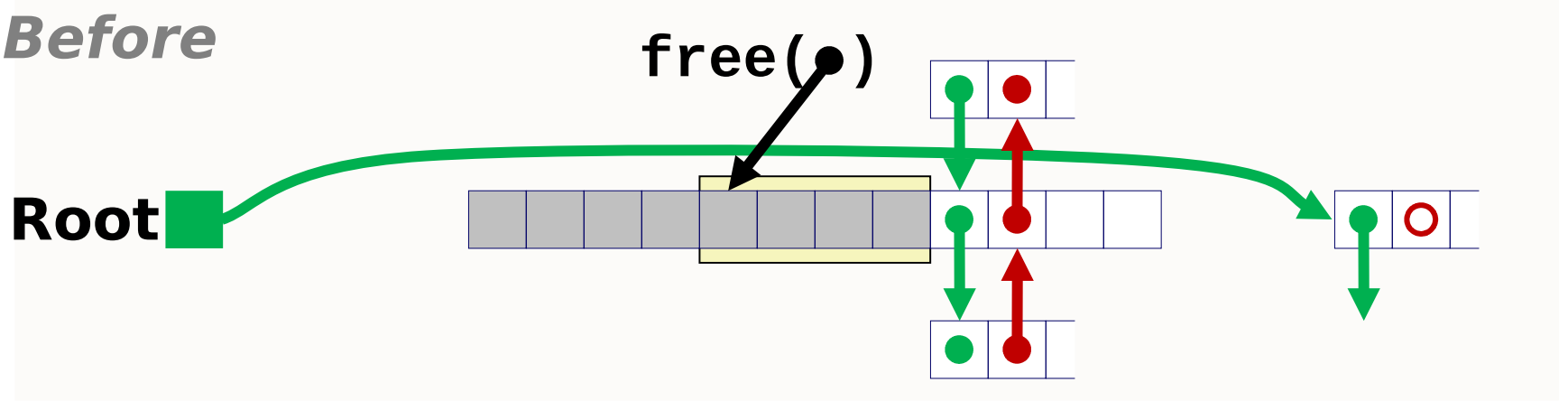
- Insert the freed block at the root of the list

After

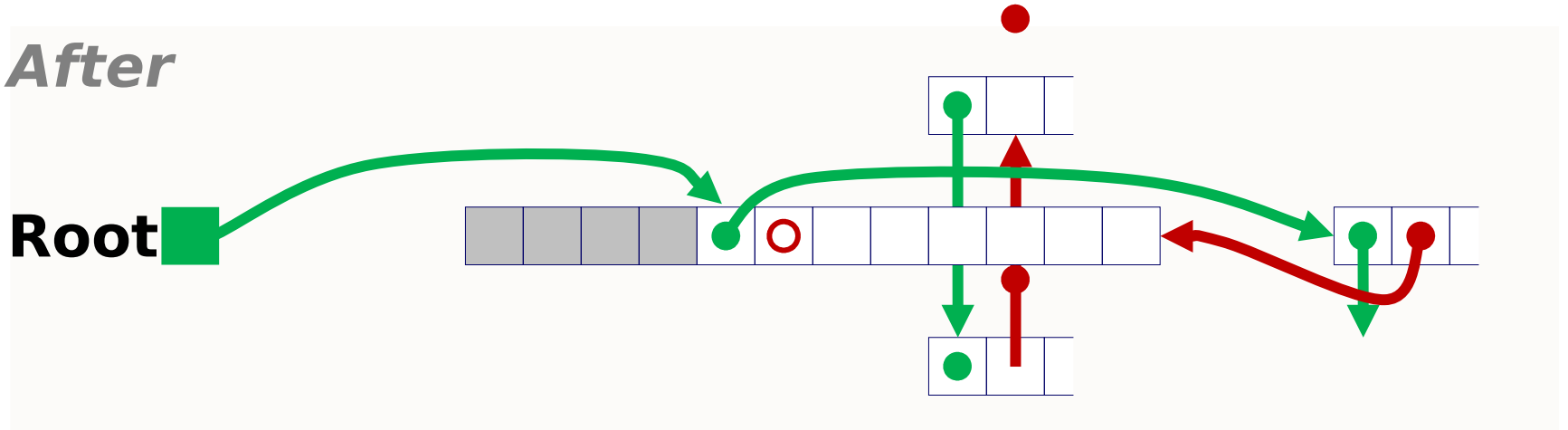


Freeing With a LIFO Policy (Case 2)

conceptual graphic

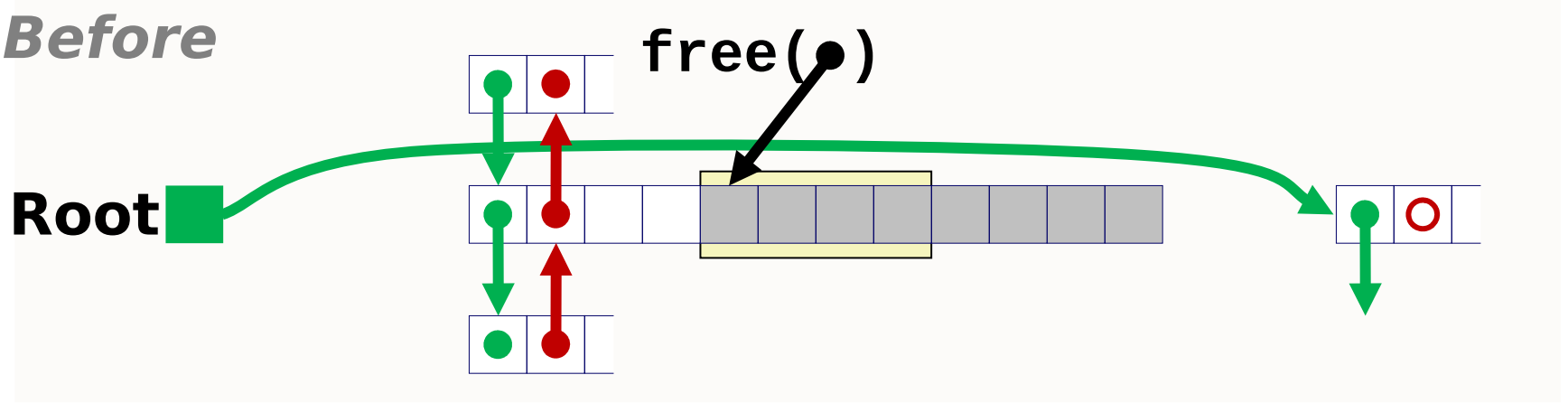


- Splice out successor block, coalesce both memory blocks and insert the new block at the root of the list

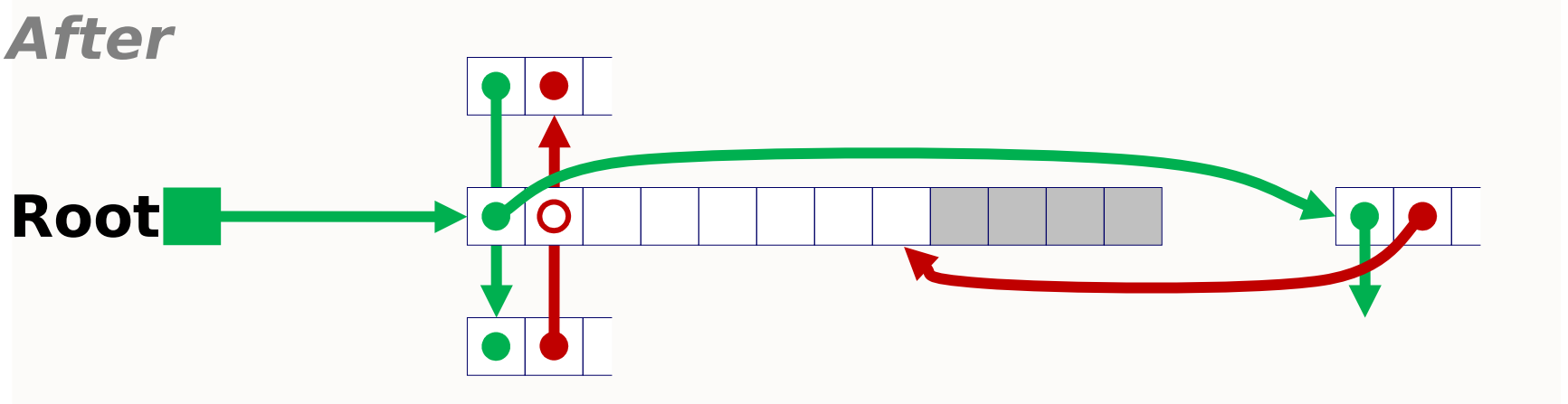


Freeing With a LIFO Policy (Case 3)

conceptual graphic

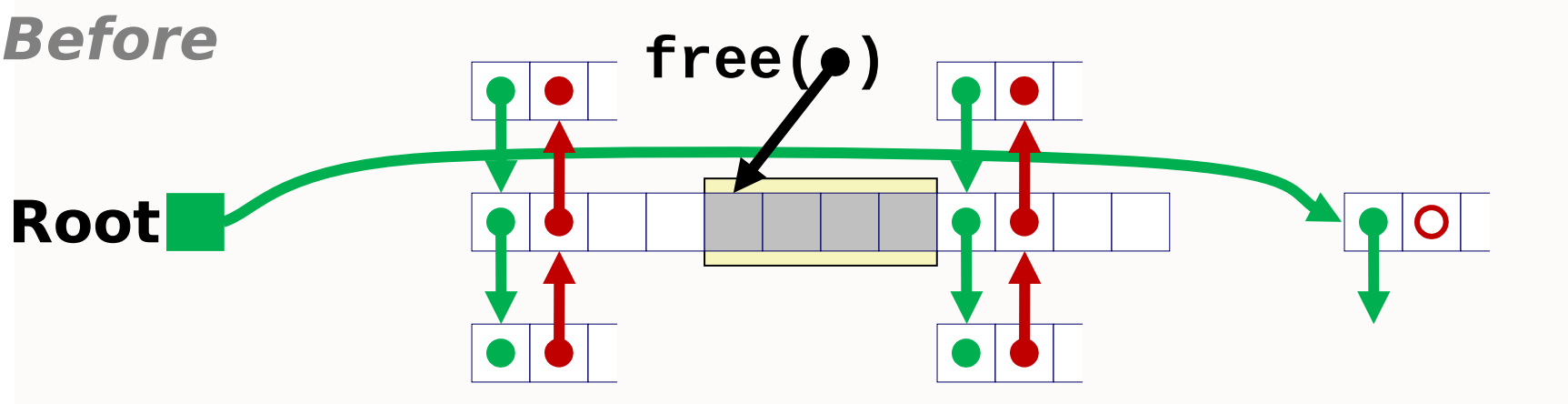


- Splice out predecessor block, coalesce both memory blocks, and insert the new block at the root of the list

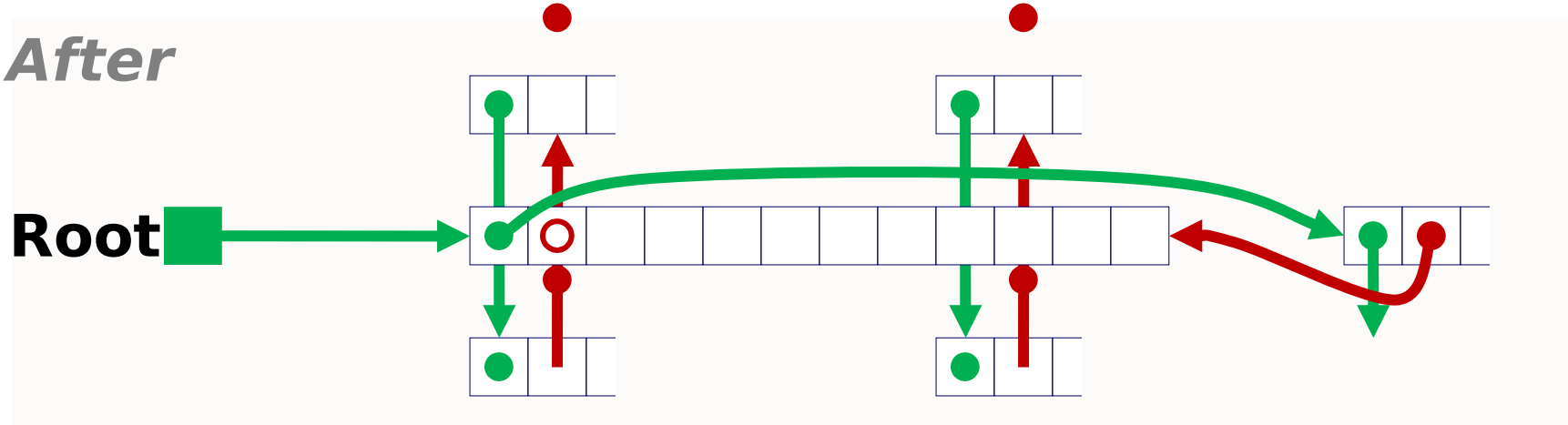


Freeing With a LIFO Policy (Case 4)

conceptual graphic



- Splice out predecessor and successor blocks, coalesce all 3 memory blocks and insert the new block at the root of the list



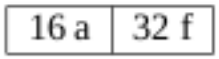
Example 2:

Consider an allocator with the following specification:

- Uses a single explicit free list.
- All memory blocks have a size that is a multiple of 8 bytes and is at least 16 bytes.
- All headers, footers, and pointers are 4 bytes in size
- Headers consist of a size in the upper 29 bits, a bit indicating if the block is allocated in the lowest bit, and a bit indicating if the previous block is allocated in the second lowest bit.
- Allocated blocks consist of a header and a payload (no footer)
- Free blocks consist of a header, two pointers for the next and previous free blocks in the free list, and a footer at the end of the block.
- All freed blocks are immediately coalesced.
- The heap starts with 0 bytes, never shrinks, and only grows large enough to satisfy memory requests.
- The heap contains only allocated blocks and free blocks. There are is no space used for other data or special blocks to mark the beginning and end of the heap.
- When a block is split, the lower part of the block becomes the allocated part and the upper part becomes the new free block.
- Any newly created free block (whether it come from a call to `free`, the upper part of a split block, or the coalescing of several free blocks) is inserted at the beginning of the free list.
- All searches for free blocks start at the head of the list and walk through the list in order.
- If a request can be fulfilled by using a free block, that free block is used. Otherwise the heap is extended only enough to fulfill the request. If there is a free block at the end of the heap, this can be used along with the new heap space to fulfill the request.

Example 2:

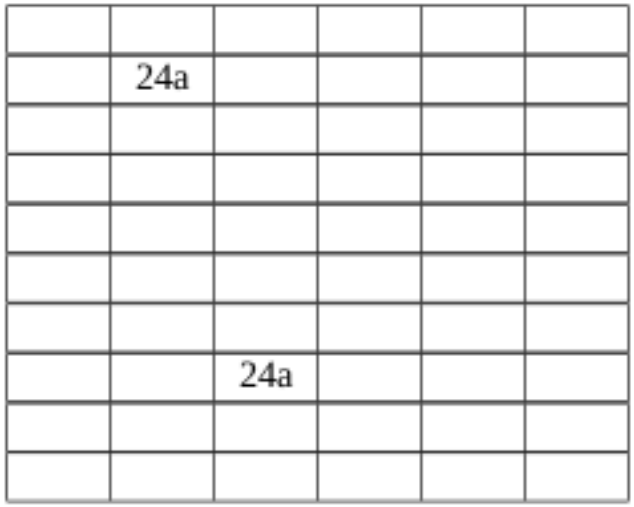
Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.



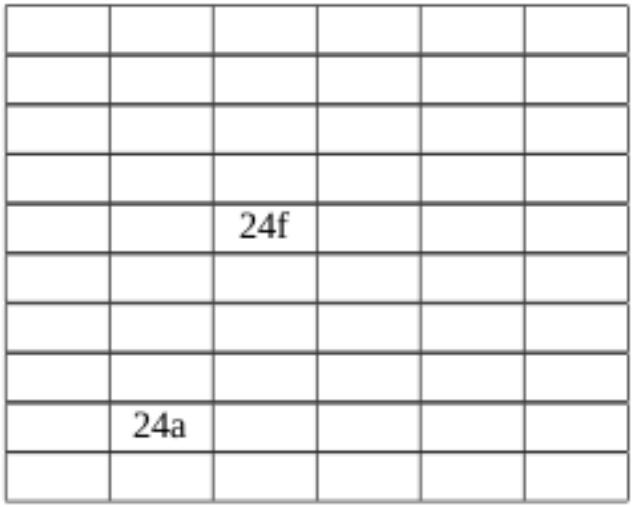
Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

```
ptr1 = malloc(32);
ptr2 = malloc(16);
ptr3 = malloc(16);
ptr4 = malloc(40);
free(ptr3);
free(ptr1);
ptr5 = malloc(16);
free(ptr4);
ptr6 = malloc(48);
free(ptr2);
```

First Fit

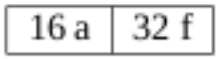


Best Fit



Example 2:

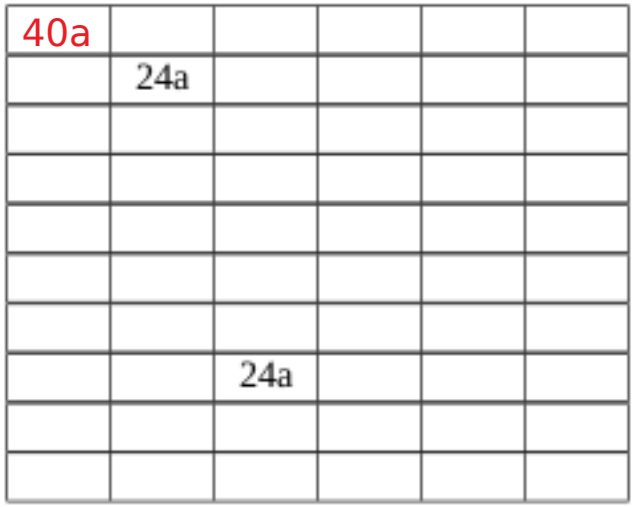
Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.



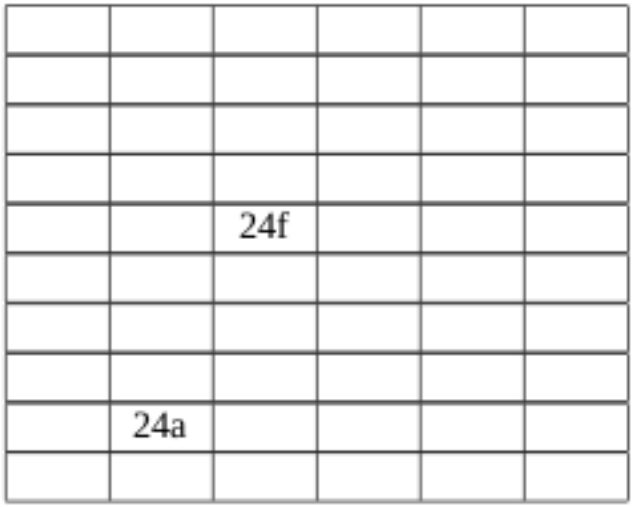
Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

```
ptr1 = malloc(32);
ptr2 = malloc(16);
ptr3 = malloc(16);
ptr4 = malloc(40);
free(ptr3);
free(ptr1);
ptr5 = malloc(16);
free(ptr4);
ptr6 = malloc(48);
free(ptr2);
```

First Fit

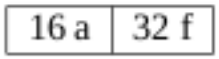


Best Fit

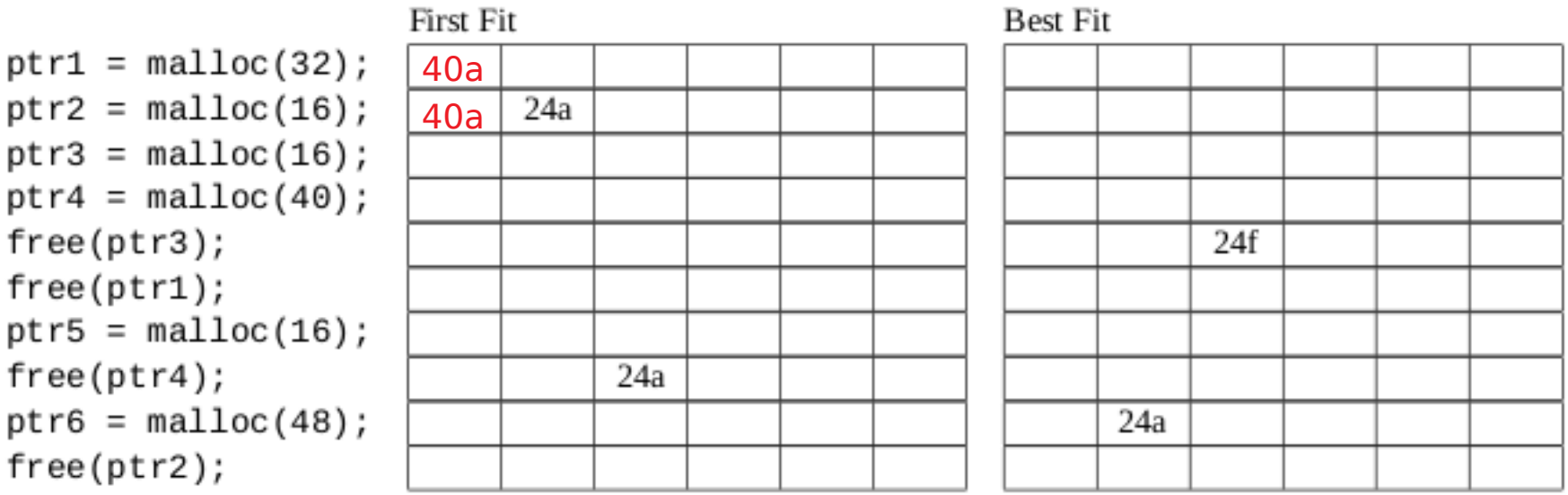


Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

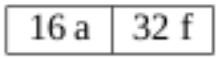


Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.



Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

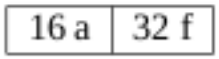


Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit	Best Fit
`ptr1 = malloc(32);`	40a	
`ptr2 = malloc(16);`	40a 24a	
`ptr3 = malloc(16);`	40a 24a 24a	
`ptr4 = malloc(40);`		
`free(ptr3);`		24f
`free(ptr1);`		
`ptr5 = malloc(16);`		
`free(ptr4);`	24a	
`ptr6 = malloc(48);`		24a
`free(ptr2);`		

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.



Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a											
`ptr2 = malloc(16);`	40a	24a										
`ptr3 = malloc(16);`	40a	24a	24a									
`ptr4 = malloc(40);`	40a	24a	24a	48a								
`free(ptr3);`									24f			
`free(ptr1);`												
`ptr5 = malloc(16);`												
`free(ptr4);`			24a									
`ptr6 = malloc(48);`								24a				
`free(ptr2);`												

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

16 a	32 f

Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a											
`ptr2 = malloc(16);`	40a	24a										
`ptr3 = malloc(16);`	40a	24a	24a									
`ptr4 = malloc(40);`	40a	24a	24a	48a								
`free(ptr3);`	40a	24a	24f	48a					24f			
`free(ptr1);`												
`ptr5 = malloc(16);`												
`free(ptr4);`			24a									
`ptr6 = malloc(48);`								24a				
`free(ptr2);`												

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

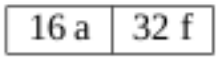
16 a	32 f

Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a											
`ptr2 = malloc(16);`	40a	24a										
`ptr3 = malloc(16);`	40a	24a	24a									
`ptr4 = malloc(40);`	40a	24a	24a	48a								
`free(ptr3);`	40a	24a	24f	48a					24f			
`free(ptr1);`	40f	24a	24f	48a								
`ptr5 = malloc(16);`												
`free(ptr4);`			24a									
`ptr6 = malloc(48);`								24a				
`free(ptr2);`												

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.



Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a											
`ptr2 = malloc(16);`	40a	24a										
`ptr3 = malloc(16);`	40a	24a	24a									
`ptr4 = malloc(40);`	40a	24a	24a	48a								
`free(ptr3);`	40a	24a	24f	48a					24f			
`free(ptr1);`	40f	24a	24f	48a								
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a							
`free(ptr4);`			24a									
`ptr6 = malloc(48);`								24a				
`free(ptr2);`												

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

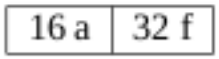
16 a	32 f

Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a											
`ptr2 = malloc(16);`	40a	24a										
`ptr3 = malloc(16);`	40a	24a	24a									
`ptr4 = malloc(40);`	40a	24a	24a	48a								
`free(ptr3);`	40a	24a	24f	48a					24f			
`free(ptr1);`	40f	24a	24f	48a								
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a							
`free(ptr4);`	24a	16f	24a	72f								
`ptr6 = malloc(48);`								24a				
`free(ptr2);`												

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

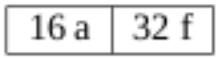


Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a											
`ptr2 = malloc(16);`	40a	24a										
`ptr3 = malloc(16);`	40a	24a	24a									
`ptr4 = malloc(40);`	40a	24a	24a	48a								
`free(ptr3);`	40a	24a	24f	48a					24f			
`free(ptr1);`	40f	24a	24f	48a								
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a							
`free(ptr4);`	24a	16f	24a	72f								
`ptr6 = malloc(48);`	24a	16f	24a	56a	16f			24a				
`free(ptr2);`												

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.



Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a											
`ptr2 = malloc(16);`	40a	24a										
`ptr3 = malloc(16);`	40a	24a	24a									
`ptr4 = malloc(40);`	40a	24a	24a	48a								
`free(ptr3);`	40a	24a	24f	48a					24f			
`free(ptr1);`	40f	24a	24f	48a								
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a							
`free(ptr4);`	24a	16f	24a	72f								
`ptr6 = malloc(48);`	24a	16f	24a	56a	16f			24a				
`free(ptr2);`	24a	40f	56a	16f								

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

16 a	32 f

Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a						40a					
`ptr2 = malloc(16);`	40a	24a										
`ptr3 = malloc(16);`	40a	24a	24a									
`ptr4 = malloc(40);`	40a	24a	24a	48a								
`free(ptr3);`	40a	24a	24f	48a					24f			
`free(ptr1);`	40f	24a	24f	48a								
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a							
`free(ptr4);`	24a	16f	24a	72f								
`ptr6 = malloc(48);`	24a	16f	24a	56a	16f			24a				
`free(ptr2);`	24a	40f	56a	16f								

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

16 a	32 f

Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a						40a					
`ptr2 = malloc(16);`	40a	24a					40a	24a				
`ptr3 = malloc(16);`	40a	24a	24a									
`ptr4 = malloc(40);`	40a	24a	24a	48a								
`free(ptr3);`	40a	24a	24f	48a					24f			
`free(ptr1);`	40f	24a	24f	48a								
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a							
`free(ptr4);`	24a	16f	24a	72f								
`ptr6 = malloc(48);`	24a	16f	24a	56a	16f			24a				
`free(ptr2);`	24a	40f	56a	16f								

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

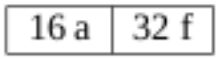
16 a	32 f

Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a						40a					
`ptr2 = malloc(16);`	40a	24a					40a	24a				
`ptr3 = malloc(16);`	40a	24a	24a				40a	24a	24a			
`ptr4 = malloc(40);`	40a	24a	24a	48a								
`free(ptr3);`	40a	24a	24f	48a					24f			
`free(ptr1);`	40f	24a	24f	48a								
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a							
`free(ptr4);`	24a	16f	24a	72f								
`ptr6 = malloc(48);`	24a	16f	24a	56a	16f			24a				
`free(ptr2);`	24a	40f	56a	16f								

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.



Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a						40a					
`ptr2 = malloc(16);`	40a	24a					40a	24a				
`ptr3 = malloc(16);`	40a	24a	24a				40a	24a	24a			
`ptr4 = malloc(40);`	40a	24a	24a	48a			40a	24a	24a	48a		
`free(ptr3);`	40a	24a	24f	48a					24f			
`free(ptr1);`	40f	24a	24f	48a								
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a							
`free(ptr4);`	24a	16f	24a	72f								
`ptr6 = malloc(48);`	24a	16f	24a	56a	16f			24a				
`free(ptr2);`	24a	40f	56a	16f								

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

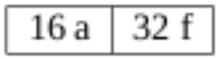
16 a	32 f

Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a						40a					
`ptr2 = malloc(16);`	40a	24a					40a	24a				
`ptr3 = malloc(16);`	40a	24a	24a				40a	24a	24a			
`ptr4 = malloc(40);`	40a	24a	24a	48a			40a	24a	24a	48a		
`free(ptr3);`	40a	24a	24f	48a			40a	24a	24f	48a		
`free(ptr1);`	40f	24a	24f	48a								
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a							
`free(ptr4);`	24a	16f	24a	72f								
`ptr6 = malloc(48);`	24a	16f	24a	56a	16f			24a				
`free(ptr2);`	24a	40f	56a	16f								

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

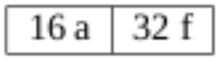


Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a						40a					
`ptr2 = malloc(16);`	40a	24a					40a	24a				
`ptr3 = malloc(16);`	40a	24a	24a				40a	24a	24a			
`ptr4 = malloc(40);`	40a	24a	24a	48a			40a	24a	24a	48a		
`free(ptr3);`	40a	24a	24f	48a			40a	24a	24f	48a		
`free(ptr1);`	40f	24a	24f	48a			40f	24a	24f	48a		
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a							
`free(ptr4);`	24a	16f	24a	72f								
`ptr6 = malloc(48);`	24a	16f	24a	56a	16f			24a				
`free(ptr2);`	24a	40f	56a	16f								

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

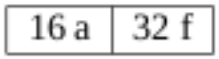


Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a						40a					
`ptr2 = malloc(16);`	40a	24a					40a	24a				
`ptr3 = malloc(16);`	40a	24a	24a				40a	24a	24a			
`ptr4 = malloc(40);`	40a	24a	24a	48a			40a	24a	24a	48a		
`free(ptr3);`	40a	24a	24f	48a			40a	24a	24f	48a		
`free(ptr1);`	40f	24a	24f	48a			40f	24a	24f	48a		
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a		40f	24a	24a	48a		
`free(ptr4);`	24a	16f	24a	72f								
`ptr6 = malloc(48);`	24a	16f	24a	56a	16f			24a				
`free(ptr2);`	24a	40f	56a	16f								

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

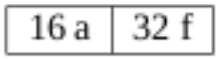


Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a						40a					
`ptr2 = malloc(16);`	40a	24a					40a	24a				
`ptr3 = malloc(16);`	40a	24a	24a				40a	24a	24a			
`ptr4 = malloc(40);`	40a	24a	24a	48a			40a	24a	24a	48a		
`free(ptr3);`	40a	24a	24f	48a			40a	24a	24f	48a		
`free(ptr1);`	40f	24a	24f	48a			40f	24a	24f	48a		
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a		40f	24a	24a	48a		
`free(ptr4);`	24a	16f	24a	72f			40f	24a	24a	48f		
`ptr6 = malloc(48);`	24a	16f	24a	56a	16f			24a				
`free(ptr2);`	24a	40f	56a	16f								

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.

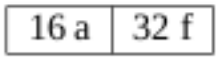


Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a						40a					
`ptr2 = malloc(16);`	40a	24a					40a	24a				
`ptr3 = malloc(16);`	40a	24a	24a				40a	24a	24a			
`ptr4 = malloc(40);`	40a	24a	24a	48a			40a	24a	24a	48a		
`free(ptr3);`	40a	24a	24f	48a			40a	24a	24f	48a		
`free(ptr1);`	40f	24a	24f	48a			40f	24a	24f	48a		
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a		40f	24a	24a	48a		
`free(ptr4);`	24a	16f	24a	72f			40f	24a	24a	48f		
`ptr6 = malloc(48);`	24a	16f	24a	56a	16f		40f	24a	24a	56a		
`free(ptr2);`	24a	40f	56a	16f								

Example 2:

Below you are given a series of memory requests. You are asked to show what the heap looks like after each request is completed using a first fit and a best fit placement policy. The heap is represented as a series of boxes, where each box is a single block on the heap, and the bottom of the heap is the left most box. In each block, you should write the total size (including headers and footers) of the block in bytes and either 'f' or 'a' to mark it as free or allocated, respectively. For example, the following heap contains an allocated block of size 16, followed by a free block of size 32.



Assume that the heap is empty before each of the sequences is run. You do not necessarily have to use all the boxes provided for the heap. Some of the boxes are already filled in to help you.

	First Fit						Best Fit					
`ptr1 = malloc(32);`	40a						40a					
`ptr2 = malloc(16);`	40a	24a					40a	24a				
`ptr3 = malloc(16);`	40a	24a	24a				40a	24a	24a			
`ptr4 = malloc(40);`	40a	24a	24a	48a			40a	24a	24a	48a		
`free(ptr3);`	40a	24a	24f	48a			40a	24a	24f	48a		
`free(ptr1);`	40f	24a	24f	48a			40f	24a	24f	48a		
`ptr5 = malloc(16);`	24a	16f	24a	24f	48a		40f	24a	24a	48a		
`free(ptr4);`	24a	16f	24a	72f			40f	24a	24a	48f		
`ptr6 = malloc(48);`	24a	16f	24a	56a	16f		40f	24a	24a	56a		
`free(ptr2);`	24a	40f	56a	16f			64f	24a	56a			